streaming: https://hackage.haskell.org/package/streaming

Of

```
-- Data.Functor.Of
data Of a b = !a :> b
instance Functor (Of a), Monoid a => Monad (Of a), Monoid a =>
Applicative (Of a),
  Foldable (Of a), Traversable (Of a),
  Bifunctor Of, Bifoldable Of, Bitraversable Of,
  ...
```

Streaming

```
data Stream f m r

-- Constructor
wrap :: (Monad m, Functor f) => f (Stream f m r) -> Stream f m r
                     -- wrap = Step        i.e. just a constructor
effect :: (Monad m, Functor f) => m (Stream f m r) -> Stream f m
r                      -- effect = Effect    i.e. just a constructor
yields :: (Monad m, Functor f) => f r -> Stream f m r
                    -- yields = wrap . (Return <$>)

replicates :: (Monad m, Functor f) => Int -> f () -> Stream f m ()
repeats :: (Monad m, Functor f) => f () -> Stream f m ()
repeatsM :: (Monad m, Functor f) => m (f ()) -> Stream f m ()
unfold :: (Monad m, Functor f) => (s -> Either r (f s)) -> s ->
Stream f m s
never :: (Monad m, Applicative f) => Stream f m r
untilJust :: (Monad m, Applicative f) => m (Maybe r) -> Stream f m
r
streamBuild :: (forall b. (r -> b) -> (m b -> b) -> (f b -> b) ->
b) -> Stream f m r
delays :: (MonadIO m, Applicative f) => Double -> Stream f m r


-- map
maps :: (Monad m, Functor f) => (forall x. f x -> g x) -> Stream f
m r -> Stream g m r
mapsPost :: (Monad m, Functor g) => (forall x. f x -> g x) ->
Stream f m r -> Stream g m r
mapsM :: (Monad m, Functor f) => (forall x. f x -> m (g x)) ->
Stream f m r -> Stream g m r
mapsMPost :: (Monad m, Functor g) => (forall x. f x -> m (g x)) ->
Stream f m r -> Stream g m r
mapped :: (Monad m, Functor f) => (forall x. f x -> m (g x)) ->
Stream f m r -> Stream g m r                                    --
same as mapsM
mappedPost :: (Monad m, Functor g) => (forall x. f x -> m (g x)) ->
Stream f m r -> Stream g m r
hoistUnexposed :: (Monad m, Functor f) => (forall a. m a -> n a) ->
Stream f m r -> Stream f n r
```

```
distribute :: (Monad m, Functor f, MonadTrans t, MFunctor t, Monad
(t (Stream f m)) => Stream f (t m) r -> t (Stream f m) r
groups :: (Monad m, Functor f) => Stream (Sum f g) m r -> Stream
(Sum (Stream f m) (Stream g m)) m r                          -- (InL
f) -> InL (Stream f m r), (InR g) -> InR (Stream g m r)


-- inspect
inspect :: Monad m => Stream f m r -> m (Either r (f (Stream f m
r)))


-- split/join
splitAt :: (Monad m, Functor f) => Int -> Stream f m r -> Stream f
m (Stream f m r)
takes :: (Monad m, Functor f) => Int -> Stream f m r -> Stream f m
()
chunksOf :: (Monad m, Functor f) => Int -> Stream f m r -> Stream
(Stream f m) m r
concats :: (Monad m, Functor f) => Stream (Stream f m) m r ->
Stream f m r
intercalates :: (Monad m, Monad (t m), MonadTrans t) => t m x ->
Stream f (t m) m r -> t m r
cutoff :: (Monad m, Functor f) => Int -> Stream f m r -> Stream f m
(Maybe r)


-- zip
zipsWith :: (Monad m, Functor h) => (forall x. f x -> g y -> h (x,
y)) -> Stream f m r -> Stream g m r -> Stream h m r
zipsWith' :: (Monad m, Functor h) => (forall x. (x -> y -> p) -> f
x -> g y -> h p) -> Stream f m r -> Stream g m r -> Stream h m r
zips :: (Monad m, Functor f, Functor g) => Stream f m r -> Stream g
m r -> Stream (Compose f g) m r
unzips :: (Monad m, Functor f) => Stream (Compose f g) m r ->
Stream f (Stream g m) r
interleaves :: (Monad m, Applicative f) => Stream f m r -> Stream f
m r -> Stream f m r
separate :: (Monad m, Functor f) => Stream (Sum f g) r -> Stream f
(Stream g m) r
unseparate :: (Monad m, Functor f) => Stream f (Stream g m) r ->
Stream (Sum f g) r
decompose :: (Monad m, Functor f) => Stream (Compose m f) m r ->
Stream f m r
expand :: (Monad m, Functor f) => (forall a b. (g a -> b) -> f a ->
h b) -> Stream f m r -> Stream g (Stream h m) r
expandPost :: (Monad m, Functor g) => (forall a b. (g a -> b) -> f
a -> h b) -> Stream f m r -> Stream g (Stream h m) r


-- fold
mapsM_ :: (Monad m, Functor f) => (forall x. f x -> m x) -> Stream
f m r -> m r
```

```
run :: Monad m => Stream m m r -> m r
destroy :: (Monad m, Functor f) => Stream f m r -> (f b -> b) -> (m
b -> b) -> (r -> b) -> m b
streamFold :: (Monad m, Functor f) => (r -> b) -> (m b -> b) -> (f
b -> b) -> Stream f m r -> m b                          -- same as
destroy
iterT :: (Monad m, Functor f) => (f (m r) -> m r) -> Stream f m r
-> r
iterTM :: (Monad m, Functor f, MonadTrans t, Monad (t m)) => (f (t
m a) -> t m a) -> Stream f (t m) r -> t m r
```

Streaming.Prelude

```
-- Of
lazily :: (a, b) -> Of a b
strictly :: Of a b -> (a, b)
_fst :: Of a b -> a
_snd :: Of a b -> b
mapOf :: (a -> b) -> Of a r -> Of b r
_first :: Functor f => (a -> f a') -> Of a b -> f (Of a' b)
_second :: Functor f => (b -> f b') -> Of a b -> f (Of a b')


-- Construct
yield :: Monad m => a -> Stream (Of a) m ()
each :: (Monad m, Foldable f) => f a -> Stream (Of a) m ()
stdinLn :: MonadIO m => Stream (Of String) m ()
readLn :: (MonadIO m, Read a) => Stream (Of a) m ()
fromHandle :: MonadIO m => Handle -> Stream (Of String) m ()
readFile :: FilePath -> (Stream (Of String) IO () -> a) -> IO a
...
```

Streaming.Internal

```
data Stream f m r =
   Step !(f (Stream f m r))
 | Effect (m (Stream f m r))
 | Return r
```

Example: Simple stream

```
import Streaming
import Streaming.Prelude qualified as S
import Data.Function


stream1 :: Stream (Of Int) IO ()          -- construct with yield
```

```
stream1 = S.yield 1                             -- first
  <> S.yield 2
  <> S.yield 3
  <> S.yield 4                                  -- last

stream2 :: Stream (Of Int) IO ()                -- construct with wrap
stream2 = return ()
  & wrap . (1 :>)                               -- last
  & wrap . (2 :>)
  & wrap . (3 :>)                               -- first

main :: IO ()
main = do
  stream1
    & S.map (*3)
    & S.filter (>4)
    & S.print

  stream2 & S.print
```

output

```
6               # stream1
9
12
3               # stream2
2
1
```

==========

streaming-bytestring

streaming-bytestring: https://hackage.haskell.org/package/streaming-bytestring

Streaming.ByteString

```
data ByteStream m r
type ByteString = ByteStream

-- Init
empty :: ByteStream m ()
singleton :: Monad m => Word8 -> ByteStream m ()
pack :: Monad m => Stream (Of Word8) m r -> ByteStream m r
unpack :: Monad m => ByteStream m r -> Stream (Of Word8) m r
fromLazy :: Monad m => ByteString -> ByteStream m ()
fromStrict :: ByteString -> ByteStream m ()
copy :: Monad m => ByteStream m r -> ByteStream (ByteStream m) r
mwrap :: m (ByteStream m r) -> ByteStream m r

toChunks :: Monad m => ByteStream m r -> Stream (Of ByteString) m r
toLazy :: Monad m => ByteStream m r -> m (Of ByteString r)
```

```
toLazy_ :: Monad m => ByteStream m r -> m ByteString
toStrict :: Monad m => ByteStream m r -> m (Of ByteString r)
toStrict_ :: Monad m => ByteStream m r -> m ByteString
effects :: Monad m => ByteStream m r -> m r
drained :: (Monad m, MonadTrans t, Monad (t m)) => t m (ByteStream
m r) -> t m r

-- Tranform
map :: Monad m => (Word8 -> Word8) -> ByteStream m r -> ByteStream
m r
for :: Monad m => ByteStream m r -> (ByteString -> ByteStream m x)
-> ByteStream m r
intercalate :: Monad m => ByteStream m () -> Stream (ByteStream m)
m r -> ByteStream m r
intersperse :: Monad m => Word8 -> ByteStream m r -> ByteStream m r

-- Operators
cons :: Monad m => Word8 -> ByteStream m r -> ByteStream m r
cons' :: Word8 -> ByteStream m r -> ByteStream m r
snoc :: Monad m => ByteStream m r -> Word8 -> ByteStream m r
append :: Monad m => ByteStream m r -> ByteStream m s -> ByteStream
m s
filter :: Monad m => (Word8 -> Bool) -> ByteStream m r ->
ByteStream m r
uncons :: Monad m => ByteStream m r -> m (Either r (Word8,
ByteStream m r))
nextByte :: Monad m => ByteStream m r -> m (Either r (Word8,
ByteStream m r))
break :: Monad m => (Word8 -> Bool) -> ByteStream m r -> ByteStream
m (ByteStream m r)
drop :: Monad m => Int64 -> ByteStream m r -> ByteStream m r
dropWhile :: Monad m => (Word8 -> Bool) -> ByteStream m r ->
ByteStream m r
group :: Monad m => ByteStream m r -> Stream (ByteStream m) m r
groupBy :: Monad m => (Word8 -> Word8 -> Bool) -> ByteStream m r ->
Stream (ByteStream m) m r
span :: Monad m => (Word8 -> Bool) -> ByteStream m r -> ByteStream
m (ByteStream m r)
splitAt :: Monad m => Int64 -> ByteStream m r -> ByteStream m
(ByteStream m r)
splitWith :: Monad m => (Word8 -> Bool) -> ByteStream m r -> Stream
(ByteStream m) m r
take :: Monad m => Int64 -> ByteStream m r -> ByteStream m ()
takeWhile :: Monad m => (Word8 -> Bool) -> ByteStream m r ->
ByteStream m ()
split :: Monad m => Word8 -> ByteStream m r -> Stream (ByteStream
m) m r
concat :: Monad m => Stream (ByteStream m) m r -> ByteStream m r
denull :: Monad m => Stream (ByteStream m) m r -> Stream
(ByteStream m) m r

-- Generate
repeat :: Word8 -> ByteStream m r
```

```
iterate :: (Word8 -> Word8) -> Word8 -> ByteStream m r
cycle :: Monad m => ByteStream m r -> ByteStream m s
unfoldM :: Monad m => (a -> Maybe (Word8, a)) -> a -> ByteStream m
()
unfoldr :: (a -> Either r (Word8, a)) -> a -> ByteStream m r
reread :: Monad m => (s -> m (Maybe ByteString)) -> s -> ByteStream
m ()

-- Fold
foldr :: Monad m => (Word8 -> a -> a) -> a -> ByteStream m () -> m
a
fold :: Monad m => (x -> Word8 -> x) -> x -> (x -> b) -> ByteStream
m r -> m (Of b r)
fold_ :: Monad m => (x -> Word8 -> x) -> x -> (x -> b) ->
ByteStream m () -> m b
head :: Monad m => ByteStream m r -> m (Of (Maybe Word8) r)
head_ :: Monad m => ByteStream m r -> m Word8
last :: Monad m => ByteStream m r -> m (Of (Maybe Word8) r)
last_ :: Monad m => ByteStream m r -> m Word8
length :: Monad m => ByteStream m r -> m (Of Int r)
length_ :: Monad m => ByteStream m r -> m Int
null :: Monad m => ByteStream m r -> m (Of Bool r)
null_ :: Monad m => ByteStream m r -> m Bool
nulls :: Monad m => ByteStream m r -> m (Sum (ByteStream m)
(ByteStream m) r)
testNull :: Monad m => ByteStream m r -> m (Of Bool (ByteStream m
r))
count :: Monad m => Word8 -> ByteStream m r -> m (Of Int r)
count_ :: Monad m => Word8 -> ByteStream m r -> m Int

-- IO
getContents :: MonadIO m => ByteStream m ()
stdin :: MonadIO m => ByteStream m ()
stdout :: MonadIO m => ByteStream m r -> m r
interact :: (ByteStream IO () -> ByteStream IO r) -> IO r
readFile :: MonadResource m => FilePath -> ByteStream m ()
writeFile :: MonadResource m => FilePath -> ByteStream m r -> m r
appendFile :: MonadResource m => FilePath -> ByteStream m r -> m r
fromHandle :: MonadIO m => Handle -> ByteStream m ()
toHandle :: MonadIO m => Handle -> ByteStream m r -> m r
hGet :: MonadIO m => Handle -> Int -> ByteStream m ()
hGetContents :: MonadIO m => Handle -> ByteStream m ()
hGetContentsN :: MonadIO m => Int -> Handle -> ByteStream m ()
hGetN :: MonadIO m => Int -> Handle -> Int -> ByteStream m ()
hGetNonBlocking :: MonadIO m => Handle -> Int -> ByteStream m ()
hGetNonBlockingN :: MonadIO m => Int -> Handle -> Int -> ByteStream
m ()
hPut :: MonadIO m => Handle -> ByteStream m r -> m r

-- Chunk operations
unconsChunk :: Monad m => ByteStream m r -> m (Either r
(ByteString, ByteStream m r))
nextChunk :: Monad m => ByteStream m r -> m (Either r (ByteString,
```

```
ByteStream m r))
chunk :: ByteString -> ByteStream m ()
foldrChunks :: Monad m => (ByteString -> a -> a) -> a -> ByteStream
m r -> m a
foldlChunks :: Monad m => (a -> ByteString -> a) -> a -> ByteStream
m r -> m (Of a r)
chunkFold :: Monad m => (x -> ByteString -> x) -> x -> (x -> a) ->
ByteStream m r -> m (Of a r)
chunkFoldM :: Monad m => (x -> ByteString -> m x) -> m x -> (x -> m
a) -> ByteStream m r -> m (Of a r)
chunkMap :: Monad m => (ByteString -> ByteString) -> ByteStream m r
-> ByteStream m r
chunkMapM :: Monad m => (ByteString -> m ByteString) -> ByteStream
m r -> ByteStream m r
chunkMapM_ :: Monad m => (ByteString -> m x) -> ByteStream m r -> m
r

-- Misc
dematerialize :: Monad m => ByteStream m r -> forall x. (r -> x) ->
(ByteString -> x -> x) -> (m x -> x) -> x
materialize :: (forall x. (r -> x) -> (ByteString -> x -> x) -> (m
x -> x) -> x) -> ByteStream m r
distribute :: (Monad m, MonadTrans t, MFunctor t, Monad (t m),
Monad (t (ByteStream m))) => ByteStream (t m) a -> t (ByteStream m)
a
zipWithStream :: Monad m => (forall x. a -> ByteStream m x ->
ByteStream m x) -> [a] -> Stream (ByteStream m) m r -> Stream
(ByteStream m) m r

toStreamingByteString :: MonadIO m => Builder -> ByteStream m ()
toStreamingByteStringWith :: MonadIO m => AllocationStrategy ->
Builder -> ByteStream m ()
toBuilder :: ByteStream IO () -> Builder
concatBuilders :: Stream (Of Builder) IO () -> Builder
```

Streaming.ByteString
Stream of **strict** ByteStrings